

Introduction to Haskell Programming

Higher-order functions: map, filter, fold

S P Suresh

Chennai Mathematical Institute

Haskell @ CMI

Higher-order functions

- Most functions produce a function as result

Higher-order functions

- Most functions produce a function as result
- We can also pass functions as arguments

Higher-order functions

- Most functions produce a function as result
- We can also pass functions as arguments
- Example: `apply f x = f x`

Higher-order functions

- Most functions produce a function as result
- We can also pass functions as arguments
- Example: `apply f x = f x`
- What is its type?

Higher-order functions

- Most functions produce a function as result
- We can also pass functions as arguments
- Example: `apply f x = f x`
- What is its type?
- A generic function `f` has type `a → b`

Higher-order functions

- Most functions produce a function as result
- We can also pass functions as arguments
- Example: `apply f x = f x`
- What is its type?
- A generic function `f` has type `a → b`
- Second argument `x` is also input to `f`

Higher-order functions

- Most functions produce a function as result
- We can also pass functions as arguments
- Example: `apply f x = f x`
- What is its type?
- A generic function `f` has type `a → b`
- Second argument `x` is also input to `f`
- Output `apply f x` is the same as `f x`

Higher-order functions

- Most functions produce a function as result
- We can also pass functions as arguments
- Example: `apply f x = f x`
- What is its type?
- A generic function `f` has type `a → b`
- Second argument `x` is also input to `f`
- Output `apply f x` is the same as `f x`
- Hence `apply :: (a → b) → a → b`

Higher-order functions

- Most functions produce a function as result
- We can also pass functions as arguments
- Example: `apply f x = f x`
- What is its type?
- A generic function `f` has type `a → b`
- Second argument `x` is also input to `f`
- Output `apply f x` is the same as `f x`
- Hence `apply :: (a → b) → a → b`
- Same as the built-in `($)`

The built-in function **map**

```
capitalize :: String → String
capitalize ""      = ""
capitalize (c:cs) = toUpper c: capitalize cs
```

```
sqrList :: [Integer] → [Integer]
sqrList []          = []
sqrList (x:xs)      = x^2 : sqrList xs
```

- Common pattern: apply a function **f** to each member in a list

The built-in function **map**

```
capitalize :: String → String
capitalize ""      = ""
capitalize (c:cs) = toUpper c: capitalize cs
```

```
sqrList :: [Integer] → [Integer]
sqrList []          = []
sqrList (x:xs)      = x^2 : sqrList xs
```

- Common pattern: apply a function **f** to each member in a list
- Built in function **map** achieves this

```
map f [x0, x1, ..., xk] ---> [f x0, f x1, ..., f xk]
```

The built-in function **map**

- Some examples

```
map (+ 3) [2,6,8] = [5,9,11]
```

```
map (* 2) [2,6,8] = [4,12,16]
```

```
map (^2) [1,2,3,4] = [1,4,9,16]
```

The built-in function **map**

- Some examples

```
map (+ 3) [2,6,8] = [5,9,11]
```

```
map (* 2) [2,6,8] = [4,12,16]
```

```
map (^2) [1,2,3,4] = [1,4,9,16]
```

- Given a list of lists, sum the lengths of inner lists

```
sumLength :: [[Int]] → Int
```

```
sumLength [] = 0
```

```
sumLength (x:xs) = length x + sumLength xs
```

The built-in function **map**

- Some examples

```
map (+ 3) [2,6,8] = [5,9,11]
```

```
map (* 2) [2,6,8] = [4,12,16]
```

```
map (^2) [1,2,3,4] = [1,4,9,16]
```

- Given a list of lists, sum the lengths of inner lists

```
sumLength :: [[Int]] → Int
```

```
sumLength [] = 0
```

```
sumLength (x:xs) = length x + sumLength xs
```

- Can be written using **map** as:

```
sumLength l = sum (map length l)
```

The built-in function `map`

- The function `map`

```
map f [] = []
```

```
map f (x:xs) = f x: map f xs
```

The built-in function `map`

- The function `map`

```
map f []      = []  
map f (x:xs) = f x: map f xs
```

- What is the type of `map`?

```
map :: (a → b) → [a] → [b]
```

The built-in function **filter**

- Select all even numbers from a list

```
allEvens :: [Int] → [Int]
allEvens [] = []
allEvens (x:xs) | even x = x: allEvens xs
                | otherwise = allEvens xs
```

The built-in function **filter**

- Select all even numbers from a list

```
allEvens :: [Int] → [Int]
allEvens [] = []
allEvens (x:xs) | even x = x: allEvens xs
                | otherwise = allEvens xs
```

- Abstract pattern:

```
filter :: (a → Bool) → [a] → [a]
filter p [] = []
filter p (x:xs) | p x = x: filter p xs
                | otherwise = filter p xs
allEvens = filter even
```

Combining `map` and `filter`

- Squares of even numbers in a list

```
sqrEvens :: [Int] → [Int]
sqrEvens l = map (^2) (filter even l)
```

Combining **map** and **filter**

- Squares of even numbers in a list

```
sqrEvens :: [Int] → [Int]
sqrEvens l = map (^2) (filter even l)
```

- Extract all vowels in a string and capitalize them

```
capVows :: String → String
capVows = map toUpper . filter isVow
isVow c = c `elem` "aeiou"
```

Combining **map** and **filter**

- Squares of even numbers in a list

```
sqrEvens :: [Int] → [Int]
sqrEvens l = map (^2) (filter even l)
```

- Extract all vowels in a string and capitalize them

```
capVows :: String → String
capVows = map toUpper . filter isVow
isVow c = c `elem` "aeiou"
```

- `(.)` denotes function composition: $(f \ . \ g) \ e = f \ (g \ e)$

The built-in function `takeWhile`

- `take n l` returns `n`-element prefix of list `l`

The built-in function `takeWhile`

- `take n l` returns `n`-element prefix of list `l`
- Instead, use a property to determine the prefix

```
takeWhile :: (a → Bool) → [a] → [a]
```

```
takeWhile (> 7) [8,1,9,10] = [8]
```

```
takeWhile (< 10) [8,1,9,10] = [8,1,9]
```

The built-in function `takeWhile`

- `take n l` returns `n`-element prefix of list `l`
- Instead, use a property to determine the prefix

```
takeWhile :: (a → Bool) → [a] → [a]
```

```
takeWhile (> 7) [8,1,9,10] = [8]
```

```
takeWhile (< 10) [8,1,9,10] = [8,1,9]
```

- `position c s` returns the first position in `s` where `c` occurs (or `length s`):

```
position c s = length (takeWhile (/= c) s)
```

The built-in function **takeWhile**

- **take** *n l* returns *n*-element prefix of list *l*
- Instead, use a property to determine the prefix

```
takeWhile :: (a → Bool) → [a] → [a]
takeWhile (> 7) [8,1,9,10] = [8]
takeWhile (< 10) [8,1,9,10] = [8,1,9]
```

- **position** *c s* returns the first position in *s* where *c* occurs (or **length** *s*):

```
position c s = length (takeWhile (/= c) s)
```

- **dropWhile** is the analogue of **drop**

zip and zipWith

- **zip** forms a list of pairs from two lists

```
zip :: [a] → [b] → [(a,b)]  
zip [] _ = []  
zip _ [] = []  
zip (x:xs) (y:ys) = (x,y):zip xs ys
```

zip and zipWith

- **zip** forms a list of pairs from two lists

```
zip :: [a] → [b] → [(a,b)]
zip [] _          = []
zip _ []          = []
zip (x:xs) (y:ys) = (x,y):zip xs ys
```

- **zipWith** combines two lists using a function

```
zipWith :: (a → b → c) → [a] → [b] → [c]
zipWith f [] _          = []
zipWith f _ []          = []
zipWith f (x:xs) (y:ys) = f x y:zipWith f xs ys
```

zip and zipWith

- **zip** forms a list of pairs from two lists

```
zip :: [a] → [b] → [(a,b)]
zip []      _      = []
zip _      []      = []
zip (x:xs) (y:ys) = (x,y):zip xs ys
```

- **zipWith** combines two lists using a function

```
zipWith :: (a → b → c) → [a] → [b] → [c]
zipWith f []      _      = []
zipWith f _      []      = []
zipWith f (x:xs) (y:ys) = f x y:zipWith f xs ys
```

- **zipWith** (+) [0,2,4,6,8] [1,3,5,7] = [1,5,9,13]

Combining elements

- Sum all numbers in a list

```
sum :: [Int] → Int
```

```
sum [] = 0
```

```
sum (x:xs) = x + sum xs
```

Combining elements

- Sum all numbers in a list

```
sum :: [Int] → Int
sum []      = 0
sum (x:xs) = x + sum xs
```

- Multiply all numbers in a list

```
product :: [Int] → Int
product []      = 1
product (x:xs) = x * product xs
```

Combining elements

- Sum all numbers in a list

```
sum :: [Int] → Int
sum []      = 0
sum (x:xs) = x + sum xs
```

- Multiply all numbers in a list

```
product :: [Int] → Int
product []      = 1
product (x:xs) = x * product xs
```

- What is the common pattern?

Combining elements

- Combining elements using `v` and `f`

```
combine :: (Int → Int → Int) → Int → [Int] → Int
combine f v []      = v
combine f v (x:xs) = f x (combine f v xs)
```

Combining elements

- Combining elements using **v** and **f**

```
combine :: (Int → Int → Int) → Int → [Int] → Int
combine f v []      = v
combine f v (x:xs) = f x (combine f v xs)
```

- Sum and product can be expressed as:

```
sum      = combine (+) 0
product = combine (∗) 1
```

foldr

- Built-in combine is called **foldr** (fold right)

```
foldr :: (a → b → b) → b → [a] → b
```

```
foldr f v [] = v
```

```
foldr f v (x:xs) = f x (foldr f v xs)
```

```
foldr f v [x1, x2, x3]
```

```
= f x1 (foldr f v [x2, x3])
```

```
= f x1 (f x2 (foldr f v [x3]))
```

```
= f x1 (f x2 (f x3 (foldr f v [])))
```

```
= f x1 (f x2 (f x3 v))
```

foldr

- **foldr** replaces `[]` by `v` and `:` by ``f`` in the list:

```
xs           = x1  : (x2  : ( ...  : xn-1  : (xn  : [])) ... )  
foldr f v xs = x1 `f` (x2 `f` ( ... `f` xn-1 `f` (xn `f` v )) ... )
```

foldr examples

- Sum and product

```
sum      = foldr (+) 0
product = foldr (*) 1

xs       = x1 : (x2 : (x3 : ( ... : xn-1 : (xn : []))))
sum xs   = x1 + (x2 + (x3 + ( ... + xn-1 + (xn + 0 ))))
product xs = x1 * (x2 * (x3 * ( ... * xn-1 * (xn * 1 ))))
```

foldr and anonymous functions

- Can express **length** in terms of **foldr**

length = **foldr** f 0 where

$f\ x\ n = n+1$

foldr and anonymous functions

- Can express **length** in terms of **foldr**

```
length = foldr f 0 where  
  f x n = n+1
```

- Not always convenient to name such functions

foldr and anonymous functions

- Can express **length** in terms of **foldr**

```
length = foldr f 0 where  
    f x n = n+1
```

- Not always convenient to name such functions
- Impedes readability sometimes

foldr and anonymous functions

- Can express **length** in terms of **foldr**

```
length = foldr f 0 where  
    f x n = n+1
```

- Not always convenient to name such functions
- Impedes readability sometimes
- **Anonymous functions:**

```
length = foldr (\x n → n+1) 0
```

Anonymous functions

- Anonymous functions are described using **lambdas**

Anonymous functions

- Anonymous functions are described using **lambdas**
- $\backslash x\ n \rightarrow n+1$ is an **unnamed** function of two arguments that increments its second argument by 1

Anonymous functions

- Anonymous functions are described using **lambdas**
- $\backslash x\ n \rightarrow n+1$ is an **unnamed** function of two arguments that increments its second argument by 1
- Can also give it a name!

Anonymous functions

- Anonymous functions are described using **lambdas**
- $\backslash x\ n \rightarrow n+1$ is an **unnamed** function of two arguments that increments its second argument by 1
- Can also give it a name!
- The two definitions of **f** below are equivalent:

```
f = \x n → n+1
```

```
f x n = n+1
```

Anonymous functions

- Anonymous functions are described using **lambdas**
- $\backslash x\ n \rightarrow n+1$ is an **unnamed** function of two arguments that increments its second argument by 1
- Can also give it a name!
- The two definitions of **f** below are equivalent:

```
f = \x n → n+1  
f x n = n+1
```

- Anonymous functions are very convenient to use with higher order functions

More `foldr` examples

- `foldr (:) []` is equivalent to the identity function on lists

More **foldr** examples

- **foldr** `(:)` `[]` is equivalent to the identity function on lists
- Let `f x l = l ++ [x]`

```
foldr f [] [x1, x2, x3]
= f x1 (foldr f [] [x2,x3])
= f x1 (f x2 (foldr f [] [x3]))
= f x1 (f x2 (f x3 (foldr f [] [])))
= f x1 (f x2 (f x3 []))
= (f x2 (f x3 [])) ++ [x1]
= ((f x3 []) ++ [x2]) ++ [x1]
= (([] ++ [x3]) ++ [x2]) ++ [x1] = [x3, x2, x1]
```

More **foldr** examples

- **foldr** `(:)` `[]` is equivalent to the identity function on lists
- Let `f x l = l ++ [x]`

```
foldr f [] [x1, x2, x3]
= f x1 (foldr f [] [x2,x3])
= f x1 (f x2 (foldr f [] [x3]))
= f x1 (f x2 (f x3 (foldr f [] [])))
= f x1 (f x2 (f x3 []))
= (f x2 (f x3 [])) ++ [x1]
= ((f x3 []) ++ [x2]) ++ [x1]
= (([] ++ [x3]) ++ [x2]) ++ [x1] = [x3, x2, x1]
```

- **foldr** `f` `[]` is just **reverse**, but takes time proportional to n^2

More **foldr** examples

- **foldr** `(:)` `[]` is equivalent to the identity function on lists
- Let `f x l = l ++ [x]`

```
foldr f [] [x1, x2, x3]
= f x1 (foldr f [] [x2,x3])
= f x1 (f x2 (foldr f [] [x3]))
= f x1 (f x2 (f x3 (foldr f [] [])))
= f x1 (f x2 (f x3 []))
= (f x2 (f x3 [])) ++ [x1]
= ((f x3 []) ++ [x2]) ++ [x1]
= (([] ++ [x3]) ++ [x2]) ++ [x1] = [x3, x2, x1]
```

- **foldr** `f` `[]` is just **reverse**, but takes time proportional to n^2
- **concat** is just **foldr** `(++)` `[]`

foldr1

- Sometimes there is no natural value to assign to the empty list

foldr1

- Sometimes there is no natural value to assign to the empty list
- For example, finding the maximum value in a list

foldr1

- Sometimes there is no natural value to assign to the empty list
- For example, finding the maximum value in a list
- Maximum is undefined for empty list

foldr1

- Sometimes there is no natural value to assign to the empty list
- For example, finding the maximum value in a list
- Maximum is undefined for empty list
- We use **foldr1** in such cases

foldr1

- Sometimes there is no natural value to assign to the empty list
- For example, finding the maximum value in a list
- Maximum is undefined for empty list
- We use **foldr1** in such cases
- Uses the last element as initial value

```
foldr1 :: (a → a → a) → [a] → a
```

```
foldr1 f [x]      = x
```

```
foldr1 f (x:xs) = f x (foldr1 f xs)
```

```
maximum = foldr1 max
```